



Retrieval Augmented Generation (RAG)

CSIT205 Generative AI – 2025



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Wollongong
Dr. Rory Sie
(rorys@uow.edu.au)

*SCHOOL OF COMPUTING AND INFORMATION
TECHNOLOGY*

*FACULTY OF ENGINEERING AND INFORMATION
SCIENCES*

Dr Andrew Yoong, SIM



UNIVERSITY
OF WOLLONGONG
AUSTRALIA



Acknowledgement of Country

We acknowledge that Country for Aboriginal peoples is an interconnected set of ancient and sophisticated relationships.

The University of Wollongong (UOW) spreads across many interrelated Aboriginal Countries that are bound by this sacred landscape, and intimate relationship with that landscape since creation. From Sydney to the Southern Highlands, to the South Coast. From fresh water to bitter water to salt. From city to urban to rural.

The University of Wollongong acknowledges the custodianship of the Aboriginal peoples of this place and space that has kept alive the relationships between all living things. The University acknowledges the devastating impact of colonisation on our campuses' footprint and commit ourselves to truth-telling, healing and education.

Journey Artwork by Wodi Wodi
Artist Lauren Henry.

The background is a vibrant rainbow with horizontal bands of red, orange, yellow, green, cyan, blue, and purple. Large, dark blue letters 'U', 'O', and 'W' are positioned on the right side of the image, partially cut off. The 'U' is at the top, the 'O' is in the middle, and the 'W' is at the bottom.

Everyone is welcome here.

UOW takes pride in
the diversity of our
community.

Find out more, seek support or join
the UOW Ally Network
uow.info/ally

Subject introduction



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Subject Description

In this subject, students delve into the world of Generative AI. They explore the foundational principles and practical applications of this cutting-edge field. Through a blend of theory and hands-on experience, students gain insights into how Generative AI empowers artificial intelligence to create original content – ranging from text and images to video and code – based on user prompts. The curriculum covers essential topics, including AI's role in code generation, effective prompt engineering techniques, and the development of advanced generative models like GANs and Transformers. Moreover, students critically examine the ethical implications of Generative AI and its impact on businesses strategy. By the end of the subject, students will have a comprehensive understanding of this promising field and the ability to responsibly apply Generative AI in real-world scenarios.

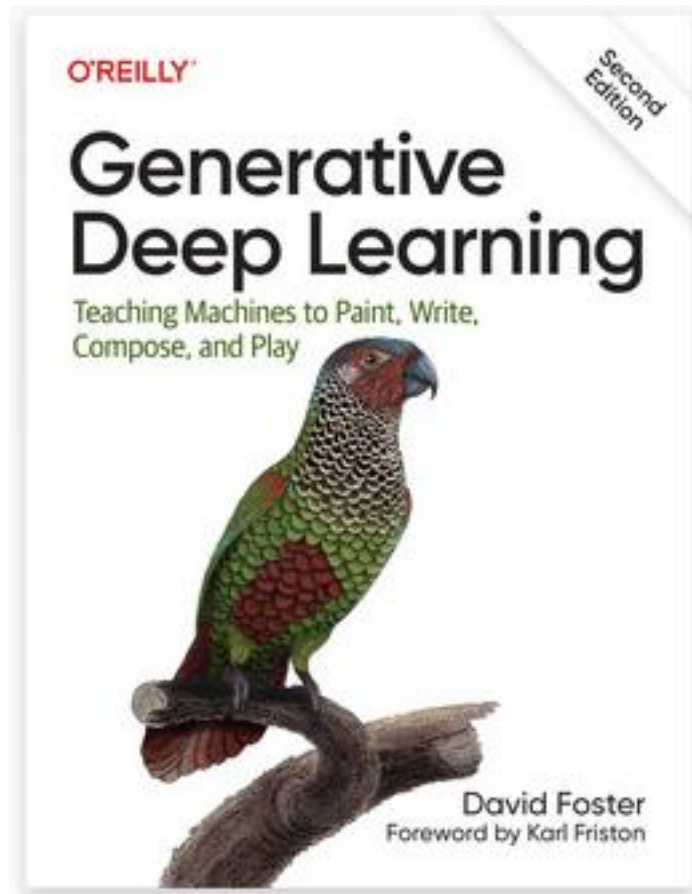
Subject Learning Outcomes (SLOs)

On successful completion of this subject, students will be able to:

1. Demonstrate an understanding of the foundations of Generative AI and its practical applications in text generation, image generation and assisted coding.
2. Apply effective prompt engineering for various tasks.
3. Demonstrate an understanding of techniques to enhance model performance, such as fine-tuning, Retrieval Augmented Generation and Reinforcement Learning using Human Feedback.
4. Identify the impact of Generative AI for industry and employ effective strategies to incorporate Generative AI.
5. Demonstrate ethical and responsible use of Generative AI.

Don't forget 

This is the book we're using



<https://www.oreilly.com/library/view/generative-deep-learning/978109813>

What did we discuss last week?



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Why do we use prompting?

- *Improved Model Performance: optimise model performance, accuracy, and efficiency.*
- *Increased Interpretability: understand how AI models process and interpret input data. By analysing the output of different prompts we gain insight into the model's strengths, weaknesses, and biases.*
- *Enhanced User Experience: impact user experience in applications like chatbots and virtual assistants. Well-crafted prompts lead to more accurate and helpful responses, improving overall usability.*

Basically we use prompting as a first attempt to enhance outcomes, before we do any retraining

Why do we use prompting?

- *Reducing Bias and Error:* mitigate bias and error in AI models by ensuring that input data is consistent and well-defined. This is essential for applications in healthcare, finance, and law.
- *Scalability and Efficiency:* scale up model performance and reduce computational resources. This enables the deployment of more complex AI models in a wide range of industries and applications.
- *Improved Explainability:* understand AI models by providing insights into how models arrive at their conclusions. This is essential for building trust in AI systems in domains like healthcare and finance.

Basically we use prompting as a first attempt to enhance outcomes, before we do any retraining

Why do we use prompting?

- *Reducing Data Overfitting: ensuring that the model generalises well to new, unseen data. This improves model robustness and adaptability in various scenarios.*

Basically we use prompting as a first attempt to enhance outcomes, before we do any retraining

Zero-shot prompting (Wei et al, 2022)

- *Instruction*
- *No example*
- *Fully relies on the LLM's pre-existing knowledge and skills*

describe this code: `def inner_product(a, b): result = 0 for i, j in zip(a, b): result += i * j return result`

Sure! Here's a detailed description of the code:

1. **Function Definition:** `def inner_product(a, b):`
 - This line defines a function named `inner_product` that takes two parameters, `a` and `b`.
2. **Initialize Result:** `result = 0`
 - This line initializes a variable `result` to 0. This variable will be used to accumulate the sum of the products of corresponding elements from `a` and `b`.
3. **For Loop with Zip:** `for i, j in zip(a, b):`
 - The `zip(a, b)` function pairs elements from `a` and `b` together. The `for` loop iterates over these pairs, with `i` taking values from `a` and `j` taking corresponding values from `b`.
4. **Accumulate Product:** `result += i * j`
 - Inside the loop, the product of `i` and `j` is calculated and added to `result`.
5. **Return Result:** `return result`
 - After the loop completes, the function returns the accumulated `result`, which is the inner product (or dot product) of the two input lists `a` and `b`.

Here's the complete code for reference:

```
def inner_product(a, b):  
    result = 0  
    for i, j in zip(a, b):  
        result += i * j  
    return result
```

This function effectively computes the inner product of two lists by multiplying corresponding elements and summing the results. Let me know if you have any questions or need further clarification!

One-shot prompting

- *Instruction*
- *One example*
- *The LLM 'learns' from one example.*
 - *This may make it more challenging to generalise to other tasks or domains.*

Task: classify the following movie review into 'positive', 'negative', or 'sarcastic'.

Input: "Masterpiece of cinema to view if you've viewed all other movies on Netflix"

Output: sarcastic

Input: "The best two hours of my day."

Output:

Output: positive

This review expresses genuine enjoyment and satisfaction, indicating that the movie provided a highly enjoyable experience for the reviewer.

Copy

Copilot

5 of 30 responses

AI-generated content may be incorrect



Few-shot prompting

- *Instruction*
- *Few examples*
- *Allows the LLM to 'learn' in-depth about how a) input is processed and b) to produce / format output.*

There is an optimal number of examples per model, but performance commonly maxes out at around 5 examples.

Task: classify the following movie review into 'positive', 'negative', or 'sarcastic'.

Input: ""Armageddon got some astronomy right. For example, there is an asteroid in the movie, and asteroids do indeed exist."

Output: sarcastic

Input: "Masterpiece of cinema to view if you've viewed all other movies on Netflix"

Output: sarcastic

Input: "Best two hours of my day"

Output: positive

Input: "Two hours of my life I'll never get back."

Output: negative

Output: positive

The review "Unforgettable experience" suggests a highly memorable and likely enjoyable time, indicating a positive opinion.

Copy

9 of 30 responses

AI-generated content may be incorrect

Copilot

Chain-of-Thought (CoT) prompting (Wei *et al.*, 2022)

Adds reasoning steps to the prompt

🏆 Achieved state-of-art performance on Math Word Problems (GSM8K) in 2022.

Four main attributes (Wei *et al.*, 2022):

- “allows models to decompose multi-step problems into intermediate steps”.
- “provides an interpretable window into the behavior of the model”.
- Can be used for math problems, commonsense reasoning and symbolic manipulation.
- Can be applied as part of few-shot prompting.

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Metrics for evaluating prompts

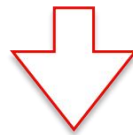
- Perplexity: A measure of how well the model predicts the next token in the sequence.
- BLEU score (Papineni et al., 2002): A metric for evaluating translation quality.
- ROUGE score (Lin, 2004): A metric for evaluating question-answering performance.
- Human evaluation: Using human evaluators to assess output quality.

Bleu

$p_n =$

$$\frac{\sum_{C \in \{Candidates\}} \sum_{n\text{-gram} \in C} Count_{clip}(n\text{-gram})}{\sum_{C' \in \{Candidates\}} \sum_{n\text{-gram}' \in C'} Count(n\text{-gram}')}.$$

$Count_{clip} = \min(Count, Max_Ref_Count)$. In other words, one truncates each word's count, if necessary, to not exceed the largest count observed in any single reference for that word.



Candidate: the the the the the the the.

Reference 1: The cat is on the mat.

Reference 2: There is a cat on the mat.

$Count(n\text{-gram}') = Count(Candidate) = 7$

$Count_{clip}(the) = \min(7, 2) = 2$

$p_n = 2/7$

Bleu Score: n-grams

- 1-grams candidate ["the", "the", ..., "the"]
- 1-grams reference ["the", "cat", "is", ..., "mat"]
- 2-grams candidate ["the the", "the the", etc.]
- 2-grams reference ["the cat", "cat is", ..., "the mat"]
- ...
- 4-grams candidate ["the the the the" x4]
- 4-grams reference ["the cat is on", "cat is on the", "is on the mat"]

Candidate: the the the the the the the.

Reference 1: The cat is on the mat.

Reference 2: There is a cat on the mat.

$$BLEU * = \prod_{n=1}^N P_n^{w_n}$$

Calculate Blue score for 1-
to N-gram and take the
average: For $N=4 \rightarrow (2/7$
 $+ 0 + 0 + 0) / 4 = 1/14$

Bleu Score: Brevity Penalty

- *Bleu* can be gamed: here it is $(2/2 + 0 + 0 + 0) / 4 = 1/4$*
- *So we introduce a Brevity Penalty for shorter candidate sentences:*
- *BLEU with a brevity penalty becomes:*

Candidate: the the

Reference 1: The cat is on the mat.

Reference 2: There is a cat on the mat.

$$BP = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases}$$

$$BLEU = BP \prod_{n=1}^N P_n^{w_n}$$

$$BP = e^{(1-r/c)} = e^{(1-6/2)} = 0.135335283237$$

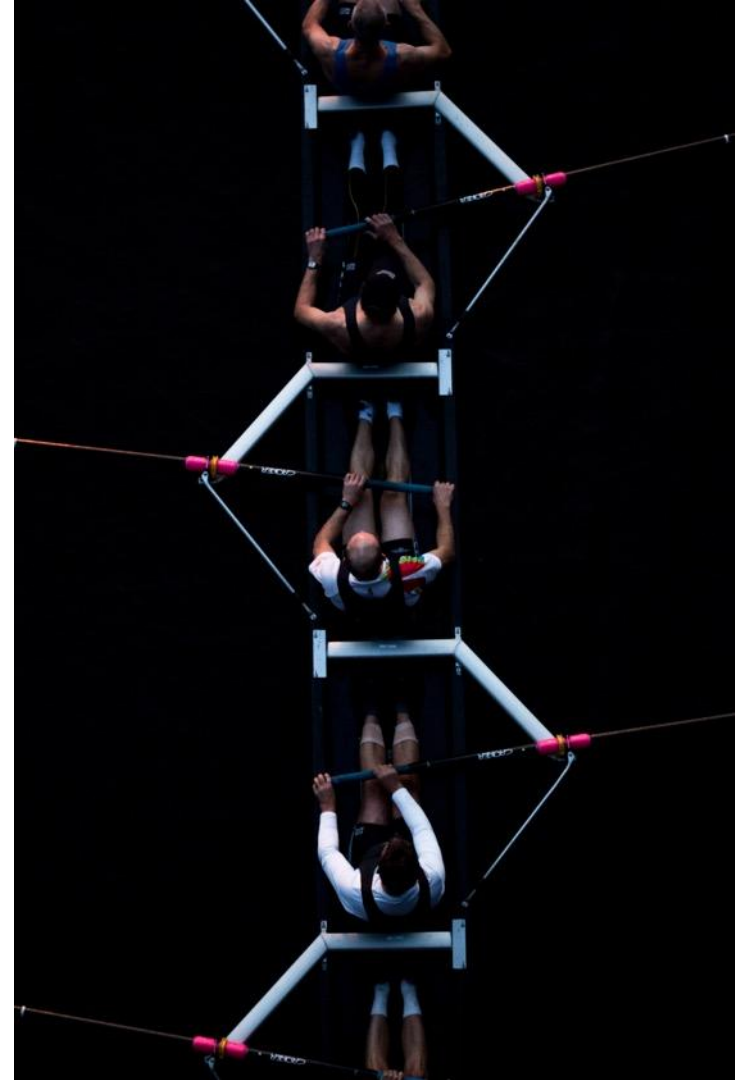
$$BLEU = 0.135335283237 * 1/4 = 0.0338338208$$

Why are we here?

CSIT205 Generative AI - Module 2

Retrieval Augmented Generation

- RAG 101
- Advanced RAGs



RAG 101



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Limitations of LLMs

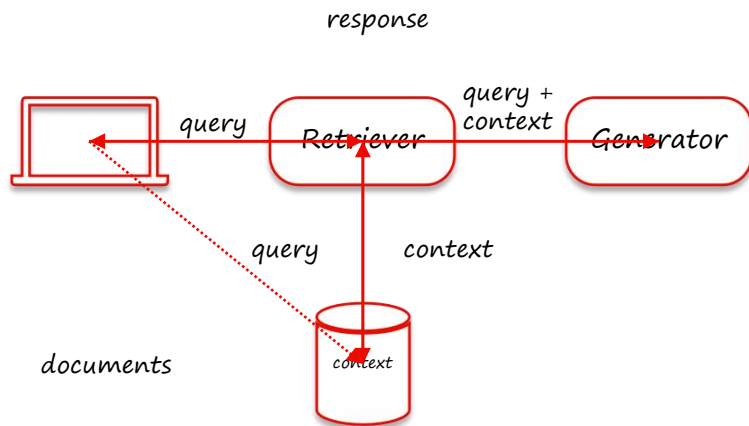
✗ No context:

- *How are you going to recommend products to a customer if you have no information about their past purchases?*
- *How are you going to recommend products to a customer if you have no information about a company's products?*

What is Retrieval Augmented Generation?

How RAG works

- Extension of Generative models to provide additional context, e.g. customers, products, business-specific knowledge.
- Retriever: retrieves additional context.
- Generator: generates a natural response like any other LLM.

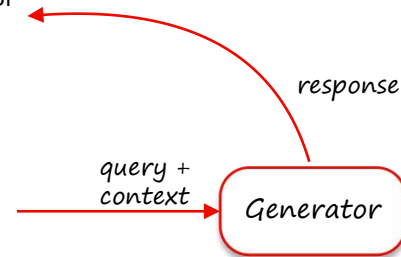


The Generator: A Classic Language Model

- A Generative model like transformers, SSMs.
- Input: prompt with instruction (query) and additional context from the Retriever.
- Output: natural language response.
- Challenge: Contextual understanding

Certainly! Let's break down and elaborate on the provided content: A Retriever-Generator Application is an AI-powered software ...

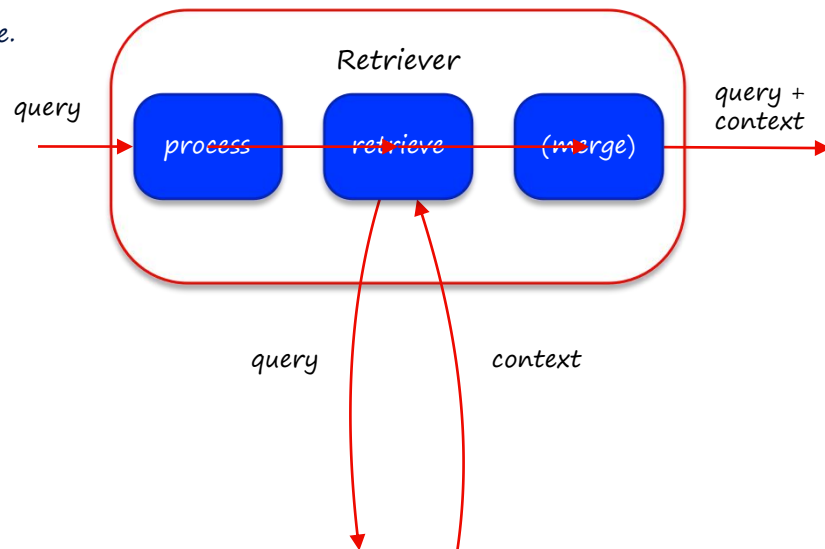
query context
Define a RAG application: A RAG (Retriever-Generator Application) is an AI-powered software system that integrates a knowledge retriever and a generator component to create a hybrid model that can efficiently search, retrieve, and generate human-readable text based on user input prompts.



The Retriever: A Knowledge Retrieval System

Searches through external knowledge bases or databases to retrieve relevant information.

- Knowledge Retrieval System: uses advanced algorithms and techniques, such as indexing, caching, and filtering, to efficiently search for relevant knowledge.
- Query processing:
 - Keyword matching
 - Entity recognition
 - Named entity disambiguation
- External knowledge retrieval (context):
 - Databases (e.g., Wikidata, DBpedia)
 - Document collections (e.g., books, articles, papers)
 - Web pages and online resources
- Query and context merge and pass to Generator: may be performed by the retriever, and depends on how much processing is needed after context is retrieved.

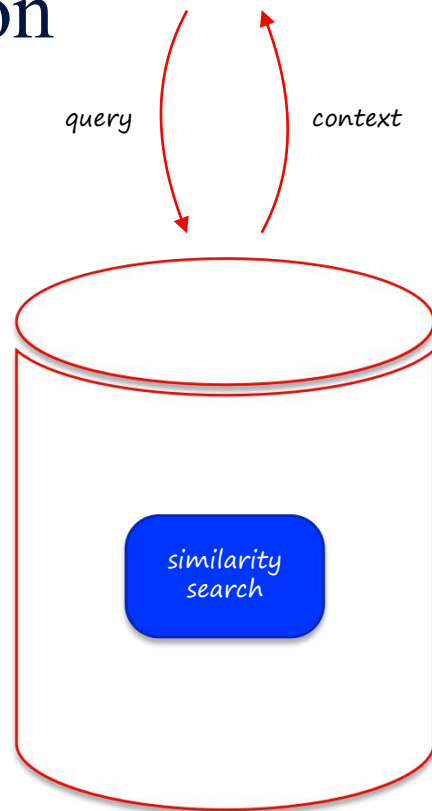


The Database: A Vector Storage Solution

Indexes information and optimises queries for efficient retrieval

- *Vector Storage: can store information in high-dimensional space, such as text, images, audio, which are represented using vectors.*
- *Similarity Search: finds similar 'documents' using vector similarity calculations such as cosine similarity, or k-nearest neighbour.*
- *Scales horizontally: because there are no relationships between documents → multiple vector databases can be searched in parallel.*

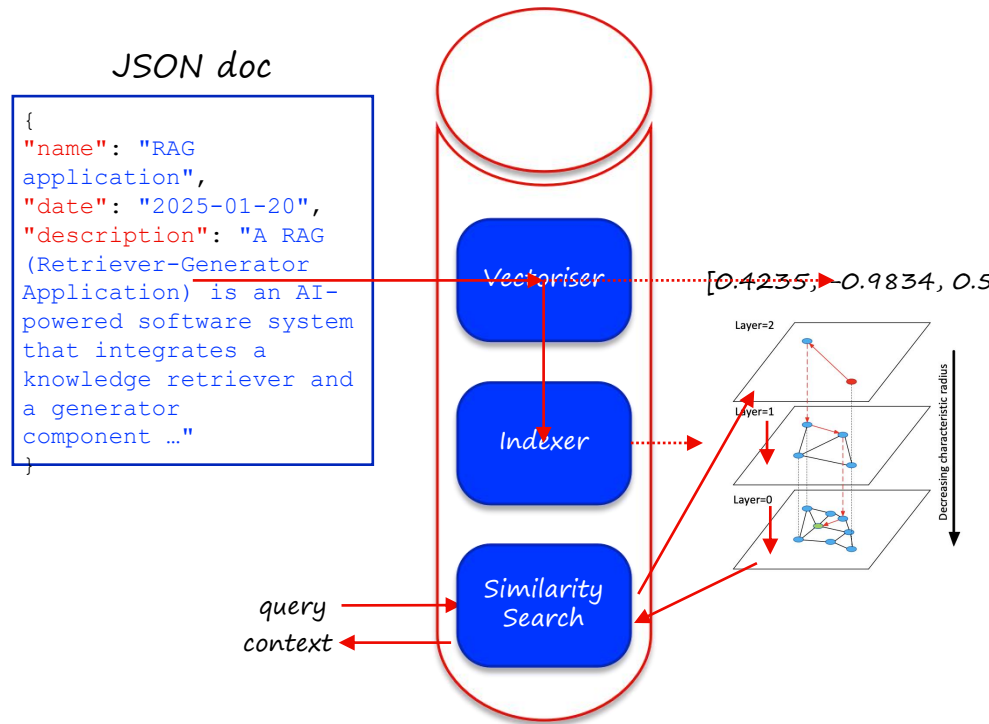
Note: Vector DBs are not the only solution, but they are the de facto standard for text, image, and audio storage / search. Alternatives include web search, RDBMS, key-value stores, etc.



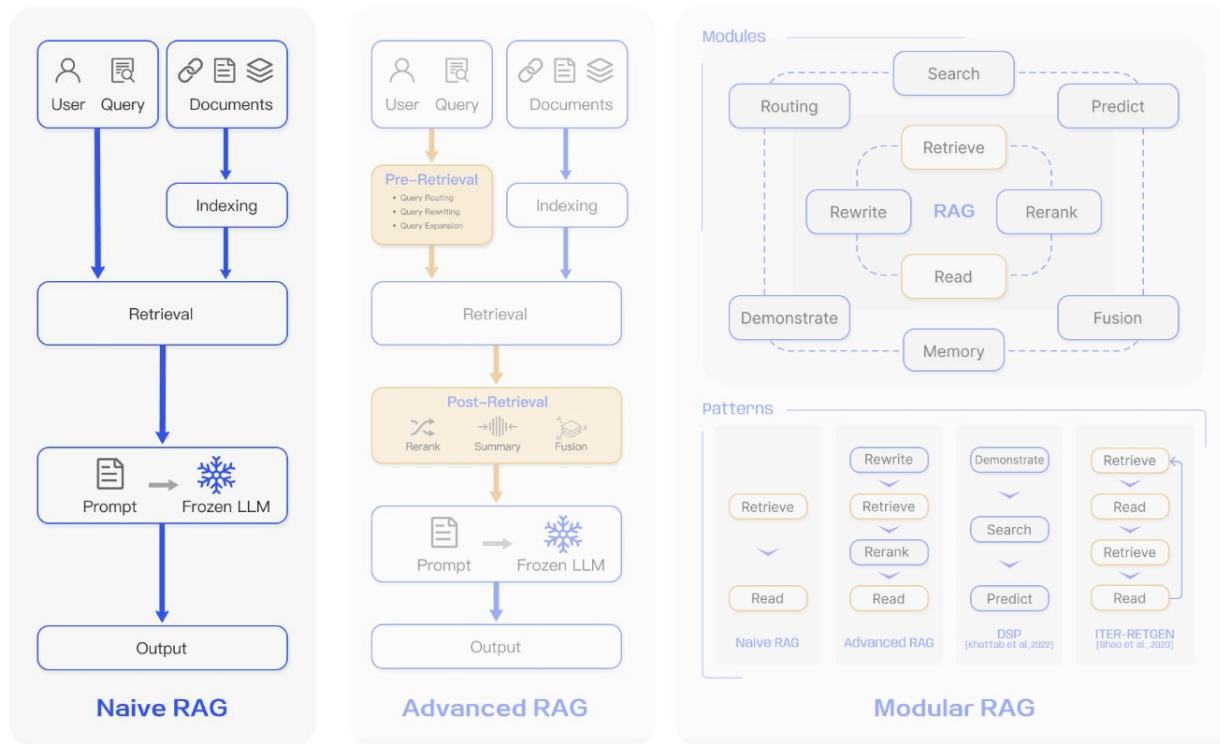
The Database: A Vector Storage Solution

How documents are stored and retrieved

- *Vectoriser: creates vectors from the given text, e.g. using text2vec, GloVe.*
- *Indexer: builds an index to retrieve vectors more efficiently, e.g. using "hierarchical navigable small world graphs" (HNSW) (Malkov & Yashunin, 2016). Uses indexes on top of indexes.*
- *Similarity Search:*
 - filter on additional properties
 - search top layer (index), then dive deeper for more detailed search.



Types of RAG applications (Gao *et al.*, 2023)



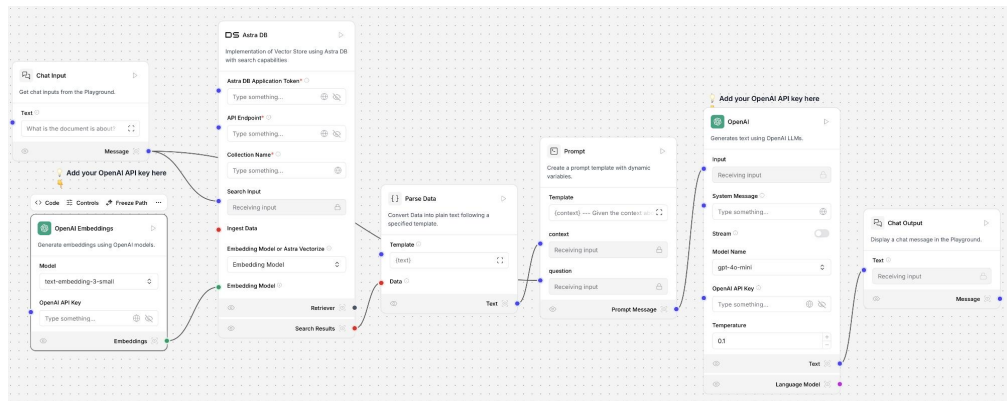
Common tools to build RAGs

• LlamaHub (by LlamaIndex)
(<https://www.llamahub.ai/>): repository of databases, integrations, agents, etc.

• Llama Packs 🐺📁 (<https://llamahub.ai/?tab=llama-packs>): readily available RAG applications and integrations.

• Langflow
(<https://www.datastax.com/products/langflow/>):
Open source low-code RAG builder.

• CrewAI (<https://www.crewai.com/open-source>):
Multi-agent platform that is quickly becoming the industry standard.



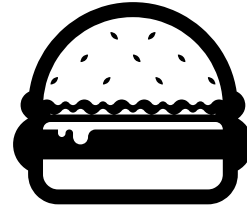
Langflow interface

10-minute Break!

This will help you learn



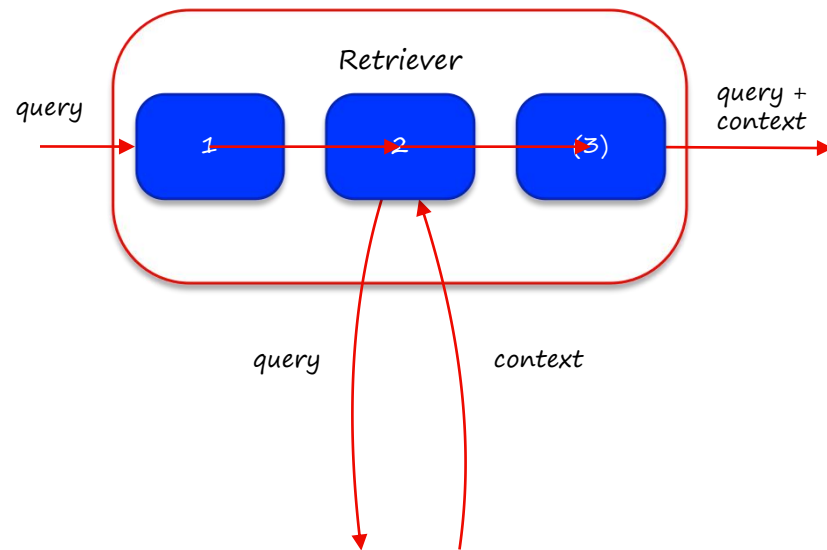
10 minutes



Fill in the boxes: Retriever architecture

Searches through external knowledge bases or databases to retrieve relevant information.

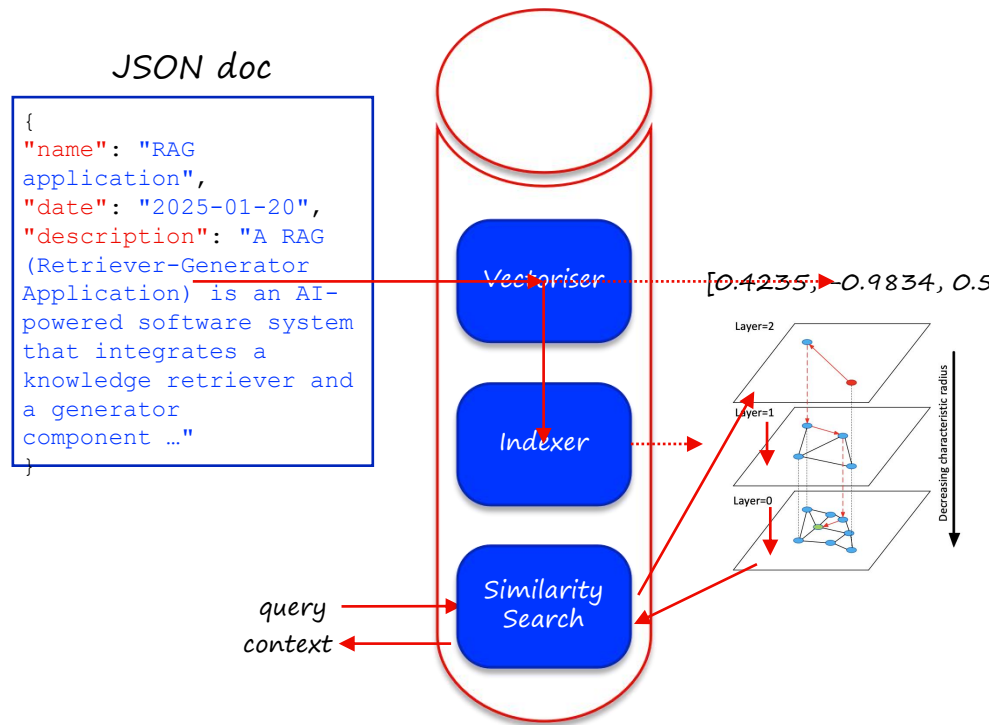
- 1.process
- 2.retrieve
- 3.merge



Fill in the blanks

How documents are stored and retrieved

- Indexer: builds an index to retrieve vectors more efficiently, e.g. using "hierarchical navigable small world graphs" (HNSW) (Malkov & Yashunin, 2016). Uses indexes on top of Vectoriser indexes.
- _____: creates vectors from the given text, e.g. using text2vec, GloVe.
- _____:
 - filter on additional properties
 - search top layer (index), then dive deeper for more detailed search.



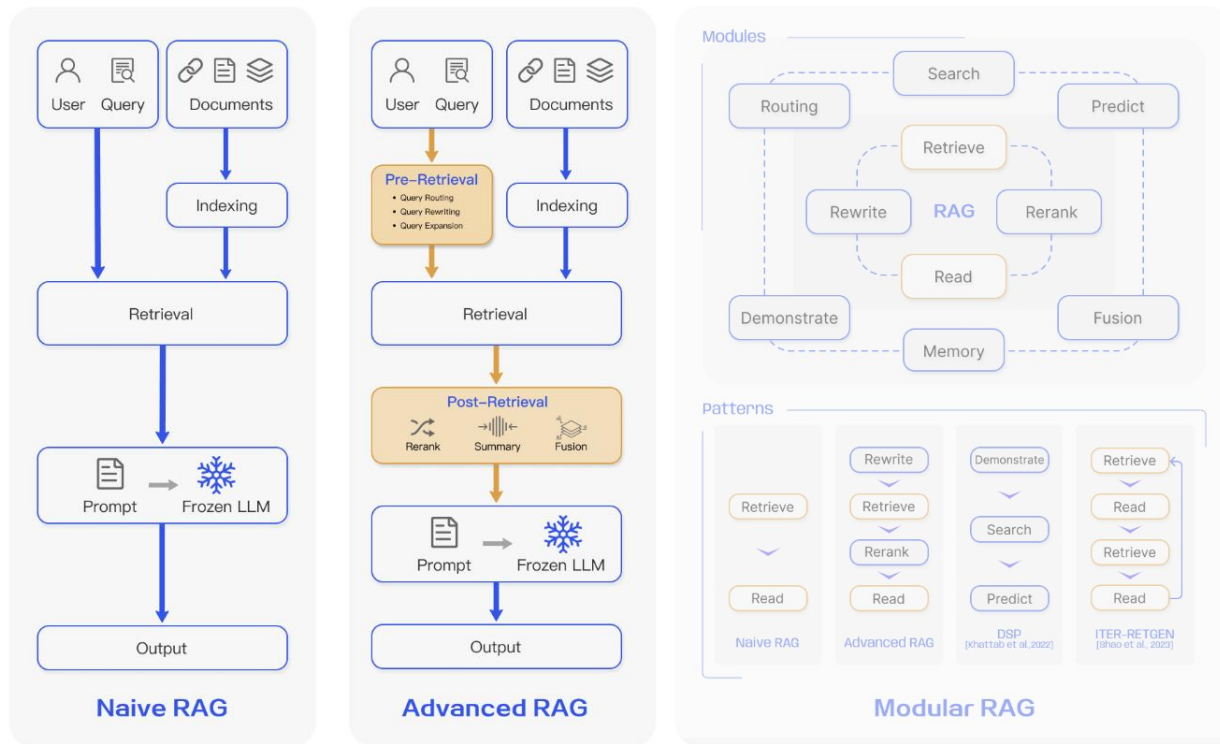
Advanced RAG



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Types of RAG applications (Gao *et al.*, 2023)

We have other types of RAG as well



Pre-Retrieval (Gao *et al.*, 2023)

Pre-processes input

- Increase Information Density Using LLMs: strip HTML / XML tags and headings, and summarise to increase information density.
- Deduplicate Information in Your Data Index Using LLMs: duplicate info makes documents less unique, which may affect their similarity score to a given query.
- Improve Retrieval Symmetry With a Hypothetical Question Index: pre-generate questions related to documents (or 'chunks'), then match the user query to that question for more efficient retrieval.

```
<!doctype html>
<html lang="en-US" prefix="og: https://ogp.me/ns#">
<head>
  <meta charset="UTF-8">
  <script>
    if (navigator.userAgent.match(/MSIE|Internet Explorer/i) ||
        navigator.userAgent.match(/Trident\/7\..*?rv:11/i)) {
      var href = document.location.href;
      if (!href.match(/[?&]nowprocket/)) {
        if (href.indexOf("?") == -1) {
          if (href.indexOf("#") == -1) {
            document.location.href = href + "?nowprocket=1"
          } else {
            document.location.href = href.replace("#", "?nowprocket=1#")
          }
        } else {
          if (href.indexOf("#") == -1) {
            document.location.href = href + "&nowprocket=1"
          } else {
            document.location.href = href.replace("#", "&nowprocket=1#")
          }
        }
      }
    }
  }
}
```



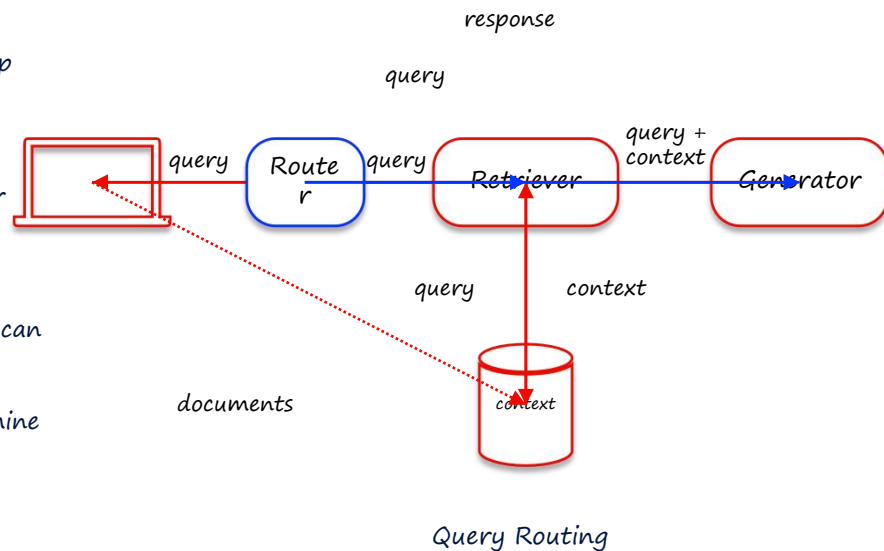
In advanced RAG, complex techniques like re-ranking, auto-merging, and advanced filtering are used to improve either the retrieval step or the generation step. The goal is to ensure that the most relevant information is retrieved in the shortest time possible. Advanced RAG techniques improve upon the efficiency, accuracy, and relevance of information retrieval and subsequent content generation. By applying these methods, you can tackle complex queries, handle diverse data sources, and create more contextually aware AI systems.

Increasing Information Density

Retrieval (Gao *et al.*, 2023)

Optimises queries and indexes for efficient retrieval

- Optimise Search Queries Using LLMs: structure the user query to aid retrieval, e.g. take a query and format it as SQL.
- Apply Hierarchical Index Retrieval: build indexes on top of indexes to speed up retrieval.
- Fix Query–Document Asymmetry With Hypothetical Document Embeddings (HyDE): Generate an answer, then use it to search the database (think vector embeddings).
- Implement Query Routing or a RAG Decider Pattern: determine if additional context is needed or not, then route to retrieval or directly to LLM. Caching can help here too.
- Self-Query Retrieval: since you can store metadata about documents, determine metadata search queries.
- Hybrid Search: combine key-word index with a vector embeddings index.
- Graph Search: knowledge can be captured in a graph structure (e.g. smoking (node) > leads to (edge) > lung cancer (node)).



Post-Retrieval (Gao *et al.*, 2023)

Processes retrieval results for quality, and compute- and cost-efficiency

- Reranking: dedicated models can rerank retrieved answers / documents to fit the user query.
- Contextual Prompt Compression: compress the retrieved answers / documents to reduce output tokens (i.e. costs).
- Score and Filter Retrieved Documents With Corrective RAG: filter retrieval results and score / classify them.
- Query Expansion: expand the query to provide a more elaborate instruction to the Generator.

"Recommend a new purchase"



"You are to recommend a new purchase. Explain why you came to this recommendation: what were their past purchases, and what did similar customers buy?"

Query Expansion

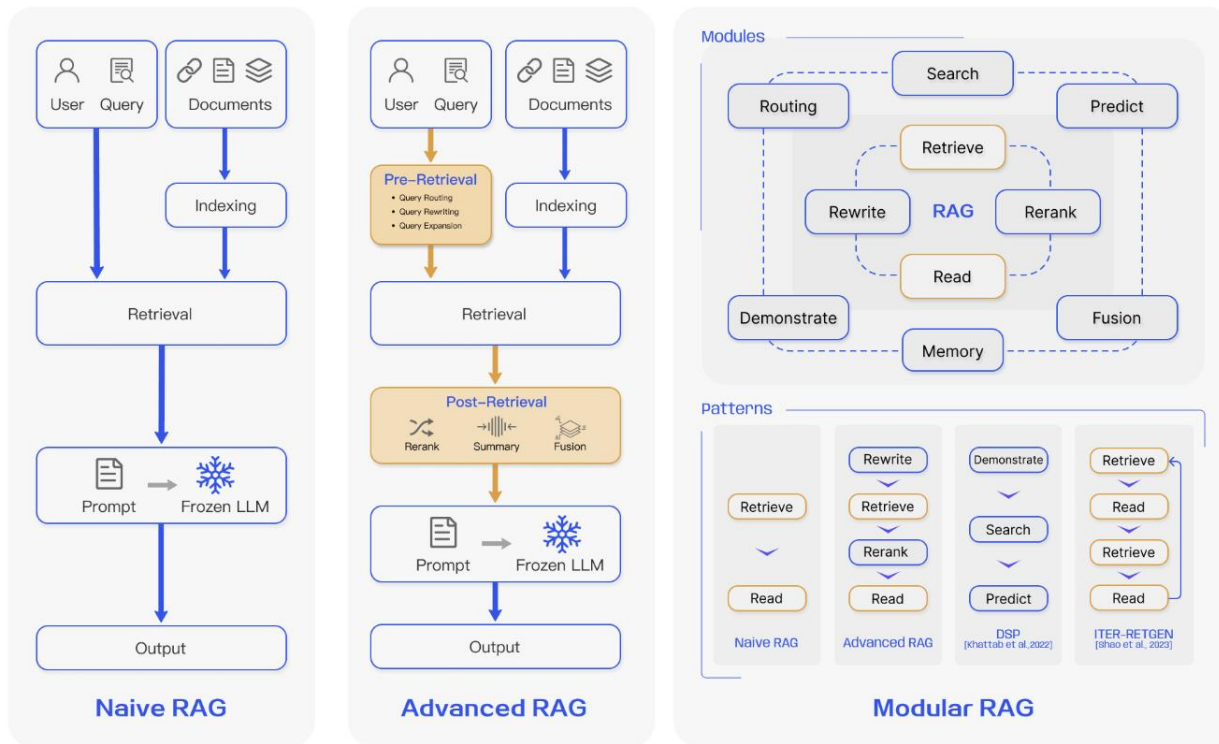
Modular RAG



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Types of RAG applications (Gao *et al.*, 2023)

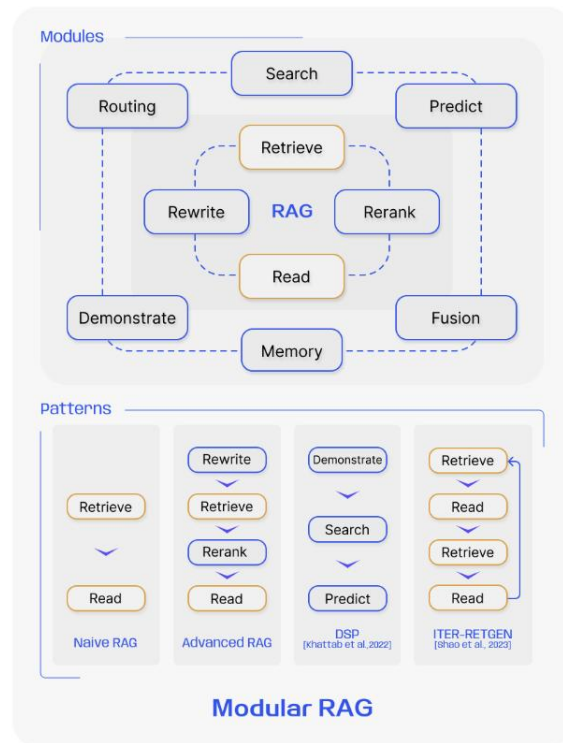
We have other types of RAG as well



Modular RAG (Gao *et al.*, 2023)

Caters for new knowledge resources, LLMs, pre- and post-retrieval modules

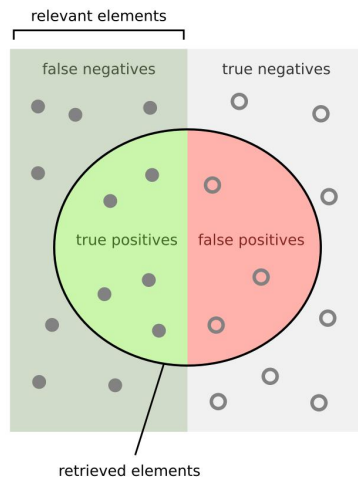
- New modules: "Additional specialised components to enhance retrieval and processing capabilities" (Gao *et al.*, 2023)
- New patterns: Modular pipelines make it easier to switch databases, LLMs, and modules.



Evaluation of RAGs

Caters for new knowledge resources, LLMs, pre- and post-retrieval modules

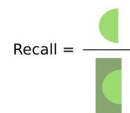
- BLEU
- ROUGE
- RaLLE (Hoshi et al., 2023):
 - R-precision: r/R where R = total number of relevant items; r = number of relevant items retrieved as top- R ;
 - Recall@5: number of items correctly recalled from a prescribed list of 5.
 - Experiments are tracked by [MLFlow](#)



How many retrieved items are relevant?



How many relevant items are retrieved?



$$R\text{-precision} = 4/5$$

$$\text{Recall@5} = 5/5$$

Candidate	Reference
A	A
C	B
B	C
D	D
F	E
E	F
G	G

Recall becomes useless when lots of documents can be retrieved

Tools to monitor LLMs

 **portkey** <https://portkey.ai/>

 **EVIDENTLY AI** <https://www.evidentlyai.com/>

 **deepchecks.** <https://www.deepchecks.com/llm-evaluation/>

Wrap-up



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Summary

- *Three RAG types: Naïve, Advanced, Modular.*
- *RAG consists of a Retriever, Generator LLM, and a knowledge resources, often vector DBs.*
- *Advanced RAG add Pre- and Post-Retrieval modules to the RAG pipeline.*
- *Modular RAG allows for new modules to be added and new patterns for RAG pipelines.*

For next lecture

- *Read Foster (2023) Chapter 9.*
- *Fine-Tuning and Reinforcement Learning with Human Feedback*

Take Home Message...

*Consider the message that you want to create when
designing your interfaces.*



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Things to think about

What is the purpose of prototyping?

*What is the difference between Lo-fi
and Hi-fi designs?*

*What is a storyboard? When can
these be used?*

What should you show your client?

References and Resources

- Gao, Yunfan, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. “Retrieval-Augmented Generation for Large Language Models: A Survey,” December 18, 2023. <http://arxiv.org/abs/2312.10997>
- Hoshi, Yasuto, Daisuke Miyashita, Youyang Ng, Kento Tatsuno, Yasuhiro Morioka, Osamu Torii, and Jun Deguchi. “RaLLe: A Framework for Developing and Evaluating Retrieval-Augmented Large Language Models.” arXiv, October 16, 2023. <https://doi.org/10.48550/arXiv.2308.10633>
- Malkov, Yu. A., & Yashunin, D. A. (2016). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. <http://arxiv.org/abs/1603.09320>
- Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL), 311–318.

Questions



UNIVERSITY
OF WOLLONGONG
AUSTRALIA